

Déploiement d'une Architecture Haute Disponibilité avec HAProxy et Nginx - Nathaniel T

1. Introduction et Objectifs

L'objectif de ce projet est de concevoir, déployer et sécuriser une infrastructure web hautement disponible au sein d'une zone démilitarisée (DMZ).

L'architecture repose sur un répartiteur de charge (Load Balancer) HAProxy qui distribue le trafic entrant vers deux serveurs web Nginx.

Le projet inclut également la sécurisation des échanges (TLS/SSL), la persistance des sessions, le filtrage réseau, la supervision des services et l'initiation à l'automatisation des déploiements.

2. Architecture Réseau (DMZ)

L'infrastructure est déployée sur le plan d'adressage suivant :

- **Passerelle (Gateway) :** 192.168.120.1
- **Masque de sous-réseau :** 255.255.255.240 (/28)
- **Serveur DNS :** 192.168.110.10
- **Serveur Répartiteur (HAProxy) :** 192.168.120.3
- **Serveur Web 1 (Nginx) :** 192.168.120.4
- **Serveur Web 2 (Nginx) :** 192.168.120.5

3. Phase 1 : Déploiement des Serveurs Web (Nginx)

3.1. Résolution de conflit de port (Nettoyage système)

Lors de l'installation initiale, un conflit sur le port TCP 80 a été détecté (erreur `bind() to 0.0.0.0:80 failed (98: Address already in use)`). Le service Apache2 préinstallé monopolisait le port.

Une purge complète a été effectuée sur les serveurs web :

```
sudo systemctl stop apache2
sudo apt purge apache2 apache2-utils apache2-bin apache2.2-
common -y
sudo apt autoremove -y
```

3.2. Installation et personnalisation de Nginx

Sur les deux serveurs (`192.168.120.4` et `192.168.120.5`), Nginx a été installé et configuré :

```
sudo apt update && sudo apt install nginx -y
sudo systemctl enable --now nginx
```

Pour identifier les serveurs lors des tests de répartition, la page d'accueil par défaut a été modifiée :

- **Sur Web1 :** `echo "<h1>Bienvenue sur le Serveur Web 1 (192.168.120.4)</h1>" | sudo tee /var/www/html/index.nginx-debian.html`
- **Sur Web2 :** `echo "<h1>Bienvenue sur le Serveur Web 2 (192.168.120.5)</h1>" | sudo tee /var/www/html/index.nginx-debian.html`

4. Phase 2 : Déploiement du Répartiteur de Charge (HAProxy)

HAProxy a été installé sur le serveur `192.168.120.3` :

```
sudo apt install haproxy -y
```

4.1. Configuration de base et gestion des Timeouts

Le fichier `/etc/haproxy/haproxy.cfg` a été configuré. Pour éviter les avertissements (`warnings`) liés à l'absence de délais d'expiration, des `timeouts` ont été définis dans la section `defaults` .

```
defaults
    mode      http
    timeout connect 5000ms
```

```
timeout client 50000ms
timeout server 50000ms
```

4.2. Sécurisation TLS (HTTPS) et Sticky Sessions

Un certificat auto-signé a été généré pour chiffrer les communications :

```
sudo mkdir -p /etc/ssl/haproxy
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -k
eyout /etc/ssl/haproxy/haproxy.key -out /etc/ssl/haproxy/ha
proxy.crt
sudo cat /etc/ssl/haproxy/haproxy.crt /etc/ssl/haproxy/hapr
oxy.key | sudo tee /etc/ssl/haproxy/haproxy.pem
```

La configuration des `frontend` et `backend` a été mise à jour pour inclure la redirection HTTPS et l'insertion de cookies pour la persistance de session (Sticky Sessions) :

```
frontend http_front
    bind *:80
    # Redirection HTTP vers HTTPS
    http-request redirect scheme https unless { ssl_fc }

    # Écoute sur le port 443 avec le certificat TLS
    bind *:443 ssl crt /etc/ssl/haproxy/haproxy.pem
    default_backend http_back

backend http_back
    balance roundrobin
    # Configuration des Sticky Sessions
    cookie SERVERID insert indirect nocache

    server web1 192.168.120.4:80 cookie web1 check
    server web2 192.168.120.5:80 cookie web2 check
```

5. Phase 3 : Durcissement et Supervision

5.1. Tableau de bord des statistiques HAProxy

Une interface de supervision a été activée pour monitorer l'état du cluster en temps réel (accessible via le port 8404).

```
listen stats
  bind *:8404
  stats enable
  stats uri /stats
  stats hide-version
  stats auth admin:SuperMotDePasse
```

5.2. Health Checks de Niveau 7 (Applicatif)

Pour éviter qu'HAProxy n'envoie du trafic vers un serveur dont le service web est planté mais le port 80 toujours ouvert, une vérification sur un fichier spécifique a été mise en place.

- **Sur Web1 et Web2 :** `echo "OK" | sudo tee /var/www/html/health.html`
- **Dans HAProxy (backend) :** Ajout de la directive `option httpchk GET /health.html`

5.3. Filtrage Réseau avec UFW

Pour empêcher l'accès direct aux serveurs web (contournement du Load Balancer), des règles de pare-feu strictes ont été appliquées sur Web1 et Web2 :

```
sudo apt install ufw -y
sudo ufw allow ssh
sudo ufw allow from 192.168.120.3 to any port 80
sudo ufw enable
```

7. Conclusion

L'architecture finale répond aux exigences de haute disponibilité, de sécurité de base et de scalabilité. Le trafic est chiffré, réparti équitablement, les sessions utilisateurs sont maintenues, et les accès directs aux backends sont bloqués. Cette infrastructure constitue une excellente base pouvant être intégrée à des pipelines de déploiement continu plus avancés (ex: Ansible, Docker).